

Global Mart Consolidation Pipeline

Documento de Análisis Técnico

Javiera Bobadilla

Junio 2026

Índice

1. Introducción	2
2. Contexto del problema	2
3. Arquitectura Propuesta	2
4. Diagrama de Flujo de Datos	3
5. Documento de Análisis Técnico	3
5.1. Contratos de Esquema	3
5.2. Orquestación del DAG	4
5.3. Matriz de Resiliencia	5
5.4. Sinking Strategy	5
5.5. Modelo de las 4 Preguntas de Apache Beam	6
5.6. Limitaciones de Implementación	7
6. Repositorio de Solución	7
7. Resultados	7
8. Conclusiones	8

1. Introducción

El presente documento describe la arquitectura propuesta para el proyecto Global Mart Consolidation Pipeline.

La solución busca consolidar información de ventas mediante un pipeline ETL basado en Apache Beam, almacenamiento columnar en Parquet, modelado analítico con dbt y orquestación mediante Apache Airflow.

El objetivo principal consiste en transformar información operacional en una capa analítica confiable, escalable y lista para consumo por herramientas de Business Intelligence.

2. Contexto del problema

Super-Market opera en distintas sedes geográficas y genera información transaccional proveniente de múltiples procesos de negocio.

Actualmente los datos presentan diversos problemas:

- información distribuida
- archivos CSV independientes
- ausencia de procesos automáticos de validación
- dificultad para consolidar historiales
- baja trazabilidad
- limitaciones para análisis posteriores

Por ello se propone una arquitectura moderna de ingeniería de datos basada en capas Silver y Gold.

3. Arquitectura Propuesta

La solución propuesta sigue una arquitectura de tipo Lakehouse, organizada en múltiples capas que permiten transformar información operacional en datos analíticos listos para la toma de decisiones.

La arquitectura se compone de las siguientes etapas:

1. **Sources:** corresponde a las fuentes de entrada del sistema, representadas por los archivos `sales_data.csv` y `status_logs.csv`.
2. **Apache Beam:** realiza el procesamiento ETL, incluyendo la lectura paralela de archivos, limpieza de registros, validación de datos, operaciones de Join mediante `CoGroupByKey` y generación de estructuras anidadas.
3. **Silver Data Lake:** almacena los resultados transformados en formato Parquet, proporcionando un esquema tipado, eficiente y optimizado para procesamiento analítico.
4. **dbt:** implementa la capa de transformación analítica, aplicando reglas de negocio, modelado dimensional y pruebas de calidad sobre los datos.
5. **PostgreSQL:** constituye la capa Gold o Serving, donde se persisten las tablas finales que serán consumidas por herramientas de Business Intelligence.
6. **Apache Airflow:** coordina la ejecución del pipeline mediante un DAG, gestionando dependencias, monitoreo, recuperación ante errores y trazabilidad del flujo de procesamiento.

Las fuentes de entrada utilizadas corresponden a:

- `sales_data.csv`
- `status_logs.csv`

4. Diagrama de Flujo de Datos

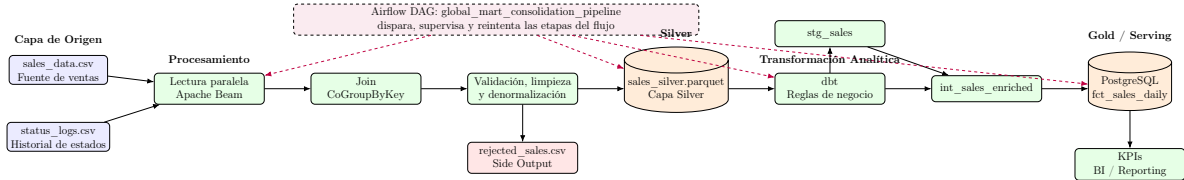


Figura 1: Diagrama de Flujo de Datos del pipeline Global Mart Consolidation Pipeline.

La figura presenta el flujo completo de procesamiento del pipeline Global Mart Consolidation Pipeline.

El proceso inicia con dos fuentes independientes de información: `sales_data.csv`, que contiene las transacciones de ventas, y `status_logs.csv`, que almacena el historial de estados asociado a cada operación.

Ambas fuentes son procesadas mediante Apache Beam, donde se realiza la lectura paralela de archivos, seguida de una operación de unión utilizando `CoGroupByKey`. Posteriormente se ejecutan tareas de validación, limpieza y denormalización de registros.

Los datos válidos son almacenados en la capa Silver mediante el archivo `sales_silver.parquet`, mientras que los registros que presentan errores son redirigidos hacia el archivo `rejected_sales.csv` mediante una estrategia de Side Outputs.

La capa Silver es consumida posteriormente por dbt, donde se aplican reglas de negocio y transformaciones analíticas para construir modelos intermedios y tablas enriquecidas.

Finalmente, la información es persistida en PostgreSQL, específicamente en la tabla `fct_sales_daily`, constituyendo la capa Gold o Serving, desde donde se generan los indicadores y métricas que pueden ser utilizados por herramientas de Business Intelligence.

De forma transversal, Apache Airflow actúa como mecanismo de orquestación, disparando, supervisando y coordinando cada una de las etapas del pipeline.

5. Documento de Análisis Técnico

5.1. Contratos de Esquema

La arquitectura propuesta implementa una transición progresiva entre distintas representaciones de los datos con el objetivo de mejorar la calidad, consistencia y capacidad analítica de la información.

Inicialmente, las fuentes de entrada corresponden a archivos CSV, los cuales presentan un esquema débil debido a la ausencia de restricciones explícitas de tipos y relaciones entre atributos.

Apache Beam realiza procesos de validación, limpieza y enriquecimiento de registros, aplicando reglas de negocio y transformaciones necesarias para garantizar la integridad de los datos.

Posteriormente, los resultados son persistidos en formato Parquet, permitiendo disponer de un esquema tipado mediante PyArrow, compresión eficiente y optimización para consultas analíticas.

Finalmente, dbt transforma la información hacia un modelo relacional implementado en PostgreSQL, constituyendo la capa Gold orientada al consumo por herramientas de Business Intelligence.

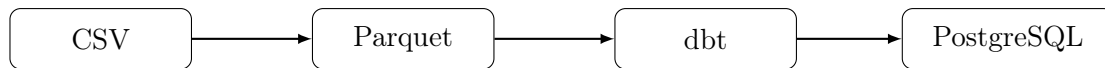


Figura 2: Transición de contratos de esquema

5.2. Orquestación del DAG

La orquestación del flujo se implementa mediante Apache Airflow, utilizando un DAG llamado `global_mart_consolidation_pipeline`. Este DAG define la secuencia lógica de ejecución del pipeline, asegurando que cada etapa se ejecute únicamente cuando la anterior finaliza correctamente.

El orden de las tareas responde a las dependencias naturales del proceso: primero se construye la capa Silver con Apache Beam, luego se verifica la existencia del archivo Parquet, posteriormente se carga la información en PostgreSQL y finalmente se ejecutan los modelos analíticos de dbt.

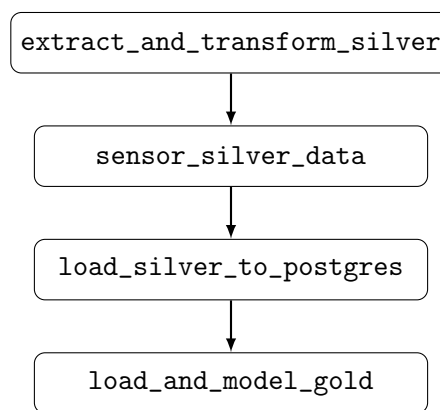


Figura 3: Secuencia de tareas del DAG implementado en Airflow.

La primera tarea, `extract_and_transform_silver`, ejecuta el pipeline de Apache Beam, encargado de leer las fuentes de entrada, aplicar validaciones, realizar el join entre ventas y estados, y generar el archivo `sales_silver.parquet`.

La tarea `sensor_silver_data` verifica que el archivo Parquet haya sido generado correctamente antes de continuar. Esta validación evita que las etapas posteriores se ejecuten sobre datos inexistentes o incompletos.

Luego, `load_silver_to_postgres` carga la información procesada hacia PostgreSQL, permitiendo que dbt disponga de una fuente estructurada para el modelado analítico.

Finalmente, `load_and_model_gold` ejecuta los comandos de dbt para construir los modelos de la capa Gold, incluyendo las tablas finales orientadas al análisis de KPIs.

Airflow aporta las siguientes capacidades al pipeline:

- trazabilidad de cada ejecución;
- monitoreo centralizado desde la interfaz web;
- control explícito de dependencias entre tareas;
- reintentos automáticos ante fallos simples;
- revisión detallada de logs;
- recuperación y depuración ante errores;
- ejecución reproducible del flujo completo.

5.3. Matriz de Resiliencia

La arquitectura implementa una estrategia de resiliencia basada en *Side Outputs* de Apache Beam, permitiendo aislar registros inválidos sin detener la ejecución del pipeline.

De esta manera, las anomalías detectadas durante la etapa de limpieza son redirigidas hacia el archivo `rejected_sales.csv`, evitando que datos inconsistentes afecten la generación de la capa Silver y el posterior modelado analítico.

Las principales excepciones consideradas se muestran en la Tabla 1.

Cuadro 1: Matriz de resiliencia del pipeline

Tipo de anomalía	Ejemplo	Acción aplicada
Monto inválido	<code>sales < 0</code>	Envío a <code>rejected_sales.csv</code>
Fecha incorrecta	<code>2026/40/80</code>	Registro descartado
Valores nulos	<code>customer_id = NULL</code>	Marcado para revisión
Duplicados	<code>transaction_id</code> repetido	Eliminación del registro

Parquet permite realizar consultas selectivas sobre columnas específicas, reduciendo operaciones de I/O y mejorando el tiempo de respuesta en comparación con formatos orientados a filas como CSV o JSON.

5.4. Sinking Strategy

La estrategia de persistencia adoptada considera el uso de archivos Parquet como formato principal de almacenamiento para la capa Silver.

La elección de este formato responde a diversos criterios técnicos relacionados con eficiencia, escalabilidad y compatibilidad con las herramientas utilizadas en la arquitectura propuesta.

Entre sus principales ventajas destacan:

- almacenamiento columnar, permitiendo la lectura selectiva de atributos;
- mecanismos de compresión nativos que reducen el espacio ocupado en disco;
- soporte para esquemas estrictos mediante PyArrow;
- compatibilidad directa con Apache Beam;
- integración con herramientas analíticas como dbt;
- optimización para consultas agregadas y consumo desde plataformas BI.

En comparación con formatos tradicionales como CSV o JSON, Parquet ofrece una mejor relación entre capacidad de almacenamiento, rendimiento de lectura y soporte para estructuras complejas.

La Tabla 2 resume las principales diferencias entre los formatos evaluados para la capa Silver.

Formato	Columnar	Compresión	Esquema	Anidado	BI
CSV	No	Baja	Débil	No	Media
JSON	No	Media	Flexible	Sí	Media
Parquet	Sí	Alta	Estricto	Sí	Alta

Cuadro 2: Comparación entre formatos de almacenamiento

En consecuencia, Parquet constituye una alternativa adecuada para la capa Silver, ya que proporciona una representación tipada, compacta y eficiente del dato, facilitando posteriormente

el modelado analítico realizado mediante dbt y la construcción de indicadores de negocio en la capa Gold.

5.5. Modelo de las 4 Preguntas de Apache Beam

El modelo de las cuatro preguntas permite describir de manera estructurada el comportamiento del pipeline desarrollado en Apache Beam, justificando las transformaciones realizadas, el entorno de ejecución y la estrategia de procesamiento implementada.

What (¿Qué?)

Define las operaciones efectuadas sobre los datos durante el proceso ETL. En el pipeline desarrollado se implementaron las siguientes transformaciones:

- lectura de las fuentes `sales_data.csv` y `status_logs.csv`;
- validación de registros y detección de anomalías;
- limpieza de datos inconsistentes;
- operaciones de unión mediante `CoGroupByKey`;
- construcción de estructuras anidadas;
- generación del archivo `sales_silver.parquet`.

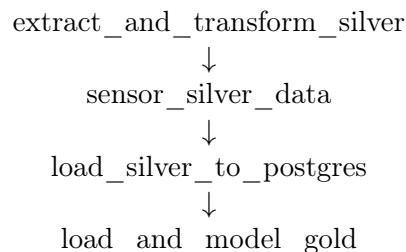
Where (¿Dónde?)

El procesamiento se ejecuta en modo Batch utilizando `DirectRunner`, permitiendo validar localmente el comportamiento del pipeline antes de migrar a entornos distribuidos como Dataflow.

La persistencia intermedia se realiza mediante archivos Parquet (Capa Silver), mientras que la información analítica final se almacena en PostgreSQL (Capa Gold).

When (¿Cuándo?)

La ejecución comienza una vez que las fuentes de entrada han sido completamente leídas. Posteriormente Airflow coordina la secuencia de ejecución a través del DAG



garantizando el cumplimiento de las dependencias entre tareas.

How (¿Cómo?)

La implementación se basa en distintas primitivas de Apache Beam:

- `ParDo`: procesamiento individual de registros;
- `CoGroupByKey`: integración de las dos fuentes de entrada;

- **Side Outputs:** separación de registros inválidos hacia `rejected_sales.csv`;
- **PyArrow:** definición de esquemas estrictos para Parquet;
- escritura columnar en formato Parquet;
- carga analítica mediante dbt y PostgreSQL.

Este enfoque permite construir un pipeline resiliente, reproducible y orientado al procesamiento analítico de datos.

5.6. Limitaciones de Implementación

Durante el desarrollo del proyecto se identificaron algunas dificultades asociadas a la integración entre herramientas ejecutadas en distintos entornos.

Airflow requirió su ejecución mediante WSL2 debido a dependencias compatibles únicamente con sistemas Linux. Adicionalmente, se presentaron incompatibilidades menores entre versiones de Python, resueltas mediante ajustes en el manejo de zonas horarias.

La principal limitación correspondió a la conectividad entre WSL y PostgreSQL ejecutado en Windows, generando errores de conexión (**Connection refused**) que impidieron completar la persistencia de datos en la capa Gold y la generación automática de KPI mediante dbt.

No obstante, la arquitectura propuesta, el pipeline en Apache Beam, la construcción de la capa Silver, el modelado analítico y la definición del DAG fueron implementados y validados satisfactoriamente.

6. Repositorio de Solución

El código fuente del proyecto se encuentra disponible en:

Repositorio GitHub del Proyecto

El repositorio contiene la implementación completa del pipeline, incluyendo Apache Beam, Airflow, dbt, PostgreSQL, el conjunto de datos de prueba y un archivo README con instrucciones de despliegue, configuración del entorno e instalación de dependencias.

7. Resultados

La solución propuesta permitió implementar una arquitectura ETL basada en Apache Beam, Parquet, dbt, PostgreSQL y Airflow.

Se logró desarrollar:

- procesamiento y validación de datos mediante Apache Beam;
- generación de la capa Silver en formato Parquet;
- definición del modelado analítico mediante dbt;
- diseño del flujo de orquestación utilizando Airflow;
- construcción del Diagrama de Flujo de Datos (DFD);
- especificación de la capa Gold para consumo analítico.

La principal limitación identificada estuvo asociada a la conectividad entre WSL y PostgreSQL, lo que impidió completar la generación de KPI sobre la capa Gold en el entorno de pruebas.

8. Conclusiones

El proyecto permitió diseñar una arquitectura moderna de ingeniería de datos compuesta por Apache Beam, Parquet, dbt, PostgreSQL y Airflow.

La propuesta incorpora principios de calidad de datos, resiliencia, orquestación y modelado analítico, siguiendo una estructura compatible con arquitecturas Lakehouse y pipelines ETL contemporáneos.

Asimismo, el desarrollo permitió identificar desafíos reales de integración entre herramientas distribuidas en distintos entornos de ejecución.

Como trabajo futuro se propone la incorporación de contenedores Docker, despliegue en la nube y servicios administrados como Dataflow o BigQuery, con el objetivo de mejorar la escalabilidad y facilitar la integración entre componentes.